PART I: DATABASE FOUNDATIONS • Chapter 4

# Entity-Relationship (ER) and Enhanced ER (EER) Modeling

## CHAPTER 4 ROADMAP & LEARNING OBJECTIVES

**Why This Matters:** *Before a civil engineer pours concrete for a bridge, they draw a precise structural blueprint. In database engineering, the Entity-Relationship (ER) and Enhanced ER (EER) diagrams are that blueprint—and converting an ER diagram into the minimum number of relational tables is one of the highest-weight topics in GATE CS, university exams, and backend system design interviews.*

**Prerequisites:** Chapters 1–3 (Schemas, Instances, Data Models, and Conceptual vs. Logical Modeling).

**Learning Objectives (What You Will Master in This Chapter):**

- Distinguish between Entity, Entity Type, and Entity Set, and classify attributes across all 8 categories (Simple, Composite, Single-Valued, Multi-Valued, Stored, Derived, Key, and Complex Nested).
- Model Unary (Recursive), Binary, and Ternary relationships, prove why a Ternary relationship is NOT equivalent to three Binary relationships, and specify Cardinality Ratios (1:1, 1:N, M:N) and Participation Constraints (Total vs. Partial / (min, max)).
- Design Weak Entity Sets with Identifying Relationships, Partial Keys (Discriminators), and multi-level weak entity chains.
- Master Enhanced ER (EER) constructs: Superclass/Subclass Specialization & Generalization (Disjoint vs. Overlapping, Total vs. Partial), Attribute/Relationship Inheritance, and Aggregation.
- Execute the complete 9-Step Algorithm for mapping ER and EER diagrams into Relational Schemas (including Options 8A–8D for subclasses).
- Solve GATE CS problems asking for the exact minimum number of relational tables required for any ER diagram (both with NULLs allowed and with zero NULLs allowed).

# Contents


4 Entity-Relationship (ER) and Enhanced ER (EER) Modeling ..... 1

4.1 Entities, Entity Sets, and the Complete Attribute Taxonomy ..... 3

4.1.1 Entity vs. Entity Type vs. Entity Set ..... 3

4.1.2 Complete Taxonomy of Attributes (Table 4.1 & Chen Symbols) ..... 3

4.1.3 Worked Example 4.1: Classifying Attributes of UniVerseDB Student ..... 4

4.2 Relationships, Degree, Roles, Cardinality, and Participation ..... 5

4.2.1 Degree of a Relationship: Unary (Recursive), Binary, and Ternary ..... 5

4.2.2 Worked Example 4.2: Proof That a Ternary Relationship != 3 Binary Relationships ..... 6

4.2.3 Cardinality Ratios (1:1, 1:N, N:1, M:N) & Participation (Total vs. Partial / (min,max)) ..... 7

4.3 Weak Entity Sets, Identifying Relationships, and Partial Keys ..... 8

4.3.1 Four Mandatory Properties of a Weak Entity Set ..... 8

4.3.2 Worked Example 4.3: Multi-Level Weak Entity Chains (BUILDING -> LAB_ROOM -> WORKS… ..... 9

4.4 Enhanced ER (EER): Specialization, Generalization, and Aggregation ..... 10

4.4.1 Disjoint (d) vs. Overlapping (o) & Total vs. Partial Specialization ..... 10

4.4.2 Aggregation: Treating a Relationship Set as a Higher-Level Entity ..... 11

4.5 Complete 9-Step Algorithm for ER- and EER-to-Relational Mapping ..... 12

4.6 GATE CS Masterclass: Minimum Number of Relational Tables for Any ER Diagram ..... 15

4.6.1 Worked Examples 4.4A, 4.4B & 4.5 (Minimum Tables & Options 8A–8D) ..... 16

4.7 End-to-End Design Walkthroughs & Notation Comparison (Chen vs. Crow’s Foot) ..... 18

4.8 Chapter 4 Summary & One-Page Rapid-Revision Cheat Sheet ..... 20

4.9 Chapter 4 Exam, GATE & Placement Question Bank (Parts A–H) ..... 22

4.10 Chapter 4 Complete Answer Key & Worked Solutions ..... 26

## 4.1 Entities, Entity Sets, and the Complete Attribute Taxonomy

Introduced by Peter Chen in 1976, the Entity-Relationship (ER) model describes the mini-world as a collection of Entities (things), Attributes (properties), and Relationships (associations).

### 4.1.1 Entity vs. Entity Type vs. Entity Set

1. Entity (Instance): A specific, distinguishable real-world object with an independent physical or conceptual existence (e.g., student Asha with Roll No 101, or course CS301 Database Systems).
2. Entity Type (Intension / Schema): A category or template defining a collection of entities that share the same attributes (e.g., STUDENT(StudentID, Name, DOB, Phone)). Drawn as a rectangle in Chen notation.
3. Entity Set (Extension / State): The actual set of all entity instances of a given Entity Type present in the database at time t.

### 4.1.2 Complete Taxonomy of Attributes

**Table 4.1: Complete Taxonomy of ER Attributes, Chen Notation, and Relational Mapping**

| Attribute Type | Definition & UniVerseDB Example | Chen ER Symbol | Relational Mapping Rule |
|---|---|---|---|
| Simple (Atomic) | Cannot be subdivided further (e.g., CGPA, Gender, Credits). | Single Oval | Becomes 1 standard column. |
| Composite | Can be divided into smaller sub-parts with independent meaning (e.g., Name -> FirstName, LastName). | Oval branching into component Ovals | Flatten into separate atomic columns (no composite column). |
| Single-Valued | Holds at most one value for a given entity (e.g., StudentID, DOB). | Single Oval | Standard column in the entity's table. |
| Multi-Valued | Can hold multiple values for a single entity (e.g., {PhoneNumbers}, {EmailIDs}). | Double-Line Oval | MUST create a separate table with (EntityPK + Attribute) as composite PK! |
| Stored vs. Derived | Stored = DOB on disk; Derived = Age computed on the fly from CurrentDate − DOB. | Dashed Oval (Derived) | Store only Stored attribute; compute Derived in View/Query. |
| Key Attribute | Minimal attribute (or set) whose value uniquely identifies each entity in the Entity Set. | Oval with Underlined text | Becomes PRIMARY KEY (or UNIQUE NOT NULL). |
| Complex (Nested) | Arbitrary nesting of Composite (...) and Multi-Valued {...} (e.g., {Address(Street, City, {Phones})}). | Nested Double & Branching Ovals | Decompose into separate normalized relation(s) or JSONB. |

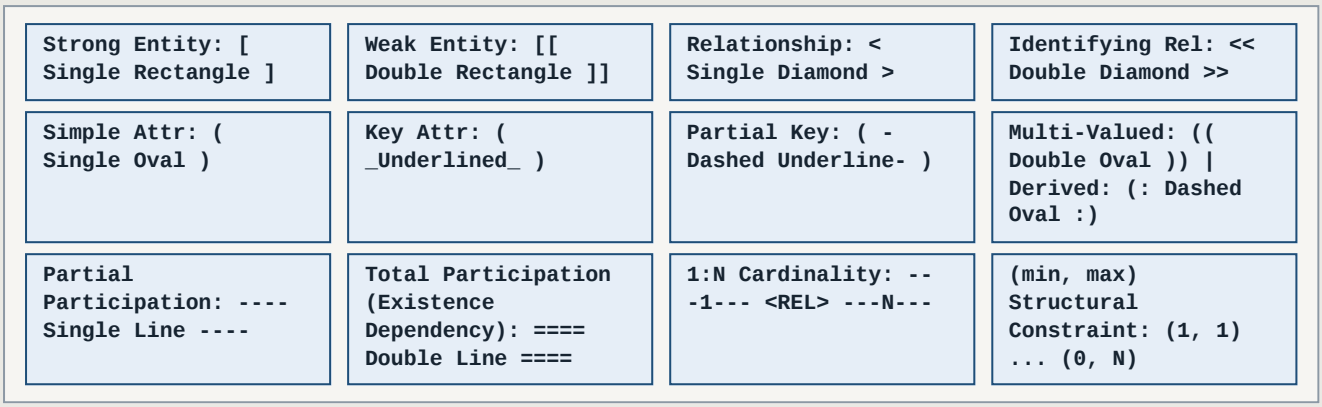



**Figure 4.1 & 4.2: Dictionary of Chen ER Diagram Symbols and Complete STUDENT — ENROLLS — COURSE Diagram.**

### 4.1.3 Worked Example 4.1: Classifying Attributes & Complex Nesting in UniVerseDB

**WORKED EXAMPLE 4.1 | Decomposing a Student Entity with All 8 Attribute Types**

Consider the entity notation for STUDENT:
STUDENT( _StudentID_, _AadhaarNo_, Name(FirstName, MiddleName, LastName), DOB, *Age*, {Phone}, {Internship(Company, City, {TechStack})} )

• Key Attributes (2 Candidate Keys): StudentID and AadhaarNo (each underlined separately!).
• Composite Attribute: Name (flattened into FirstName, MiddleName, LastName in the Student table).
• Stored vs. Derived: DOB is Stored; *Age* is Derived (dashed oval, not stored as a mutable column).
• Multi-Valued Attribute: {Phone} -> Mapped to a separate table Student_Phone(StudentID FK, Phone, PRIMARY KEY(StudentID, Phone)).
• Complex Attribute: {Internship(Company, City, {TechStack})} combines multi-valued {...} and composite (...) -> Mapped to separate child tables!

**COMMON MISTAKE | Two Separate Underlines vs. One Shared Composite Underline**

If an entity has TWO separate underlined ovals (_StudentID_ and _Email_), it has TWO distinct Candidate Keys. If it has a composite key (e.g., _CourseID_ + _SectionNo_), both components share a single composite primary key!

## 4.2 Relationships, Degree, Roles, Cardinality, and Participation

### 4.2.1 Degree of a Relationship: Unary (Recursive), Binary, and Ternary

The degree of a relationship type is the number of participating entity types:

1. Unary (Recursive) Relationship (Degree = 1): Connects instances of the SAME entity set in distinct Role Names. Examples: STUDENT —< MENTORS >— STUDENT (with roles Senior_Mentor and Junior_Mentee, 1:N) or COURSE —< PREREQUISITE_OF >— COURSE (with roles PrereqCourse and AdvancedCourse, M:N). Role labels on the connecting lines are mandatory in Unary relationships!
2. Binary Relationship (Degree = 2): Connects two entity sets (the most common case, e.g., DEPARTMENT —< EMPLOYS >— FACULTY).
3. Ternary Relationship (Degree = 3): Simultaneously connects three entity sets in a single atomic fact—for example, SUPPLIER, PART, and PROJECT linked by < SUPPLIES(Quantity) >, recording that Supplier s supplies Part p to Project j.

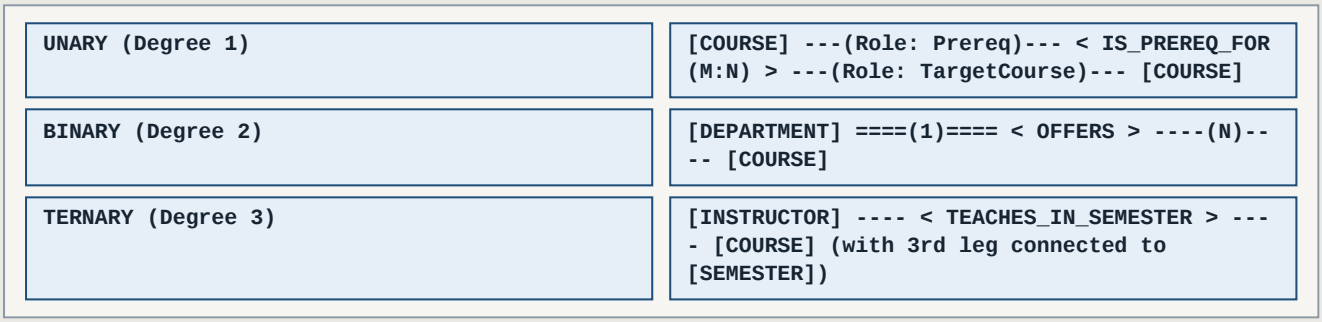



**Figure 4.3: Unary (Recursive with Role Labels), Binary, and Ternary Relationship Diagrams.**

### 4.2.2 Worked Example 4.2: Proof That a Ternary Relationship != Three Binary Relationships

**WORKED EXAMPLE 4.2 | Counterexample Proof (Spurious Tuple Generation)**

Suppose we want to record which Instructor teaches which Course in which Semester: Ternary relation TEACHES(Instructor, Course, Semester).
Can we replace TEACHES with three binary relationships: CAN_TEACH(Instructor, Course), OFFERED_IN(Course, Semester), and ACTIVE_IN(Instructor, Semester)?

Proof by Counterexample:
Suppose our true real-world facts in TEACHES are only TWO tuples:
• Fact 1: (Prof. Sharma, CS301, Fall_2026)
• Fact 2: (Prof. Verma, CS301, Spring_2027)
• Fact 3: (Prof. Sharma, CS201, Spring_2027)

If we decompose into the three binary tables:
• CAN_TEACH contains (Prof. Sharma, CS301), (Prof. Verma, CS301), (Prof. Sharma, CS201).
• OFFERED_IN contains (CS301, Fall_2026), (CS301, Spring_2027), (CS201, Spring_2027).
• ACTIVE_IN contains (Prof. Sharma, Fall_2026), (Prof. Verma, Spring_2027), (Prof. Sharma, Spring_2027).

Now take the Natural Join CAN_TEACH ⋈ OFFERED_IN ⋈ ACTIVE_IN:
Because (Prof. Sharma, CS301) ∈ CAN_TEACH, (CS301, Spring_2027) ∈ OFFERED_IN, and (Prof. Sharma, Spring_2027) ∈ ACTIVE_IN, the join produces the SPURIOUS (FALSE) TUPLE:
(Prof. Sharma, CS301, Spring_2027) — even though Prof. Verma taught CS301 in Spring_2027!
Therefore, a Ternary relationship carries joint 3-way information that CANNOT be reconstructed losslessly from three binary relationships!

### 4.2.3 Cardinality Ratios & Participation Constraints (Total vs. Partial / (min, max))

Structural constraints on a relationship combine two orthogonal rules:

1. Cardinality Ratio (Maximum Participation — 1:1, 1:N, N:1, M:N): Specifies the maximum number of relationship instances an entity can participate in. For example, DEPARTMENT (1) —< HAS_HOD >— (1) FACULTY is 1:1; DEPARTMENT (1) —< EMPLOYS >— (N) FACULTY is 1:N; STUDENT (M) —< ENROLLS >— (N) COURSE is M:N.
2. Participation Constraint (Minimum Participation — Total vs. Partial): Specifies whether the existence of an entity depends on its being related to another entity via the relationship:
   • Total Participation (Mandatory, min >= 1, drawn as a DOUBLE LINE ====): Every entity in the set MUST participate in at least one relationship instance (e.g., every COURSE must be offered by a DEPARTMENT).
   • Partial Participation (Optional, min = 0, drawn as a SINGLE LINE ----): Some entities may not participate (e.g., not every FACULTY member is a Head of Department).

**EXAM TRAP | Reversal of Sides Between 1:N Chen Labels and (min, max) Notation!**

In standard (min, max) notation placed on the line attached to Entity E, (min, max) states how many relationship instances EACH instance of E participates in:
• In DEPARTMENT (1) ==== <OFFERS> ---- (N) COURSE:
- Each DEPARTMENT offers at least 1 and at most N courses -> label on DEPARTMENT side is (1, N).
- Each COURSE belongs to exactly 1 department -> label on COURSE side is (1, 1)!
Notice that the max values (N and 1) appear on the OPPOSITE sides compared to traditional 1:N labels!

## 4.3 Weak Entity Sets, Identifying Relationships, and Partial Keys

Not every entity has its own globally unique Primary Key. Consider a Course SECTION (Section 1, Section 2) or a Hostel ROOM (Room 101): "Section 1" means nothing unless you know whether it is Section 1 of CS301 or Section 1 of EC204!

### 4.3.1 Four Mandatory Properties of a Weak Entity Set

1. No Primary Key of Its Own: A Weak Entity Set (drawn as a double rectangle [[ SECTION ]]) lacks sufficient attributes to form a globally unique primary key.
2. Owner (Identifying) Strong Entity Set: It exists only in relation to an Identifying Owner Entity Set (e.g., [ COURSE ]) via an Identifying Relationship (drawn as a double diamond << HAS_SECTION >>).
3. Always Total Participation & 1:N (or 1:1) from Owner: The Weak Entity Set ALWAYS has Total Participation (double line ====) in its identifying relationship because a weak entity cannot exist without its owner (Existence Dependency).
4. Partial Key (Discriminator): A Weak Entity Set has a Partial Key (drawn with a dashed underline, e.g., -SecNo- and -Semester-) that uniquely distinguishes weak entities belonging to the SAME owner. Its full relational Primary Key is:

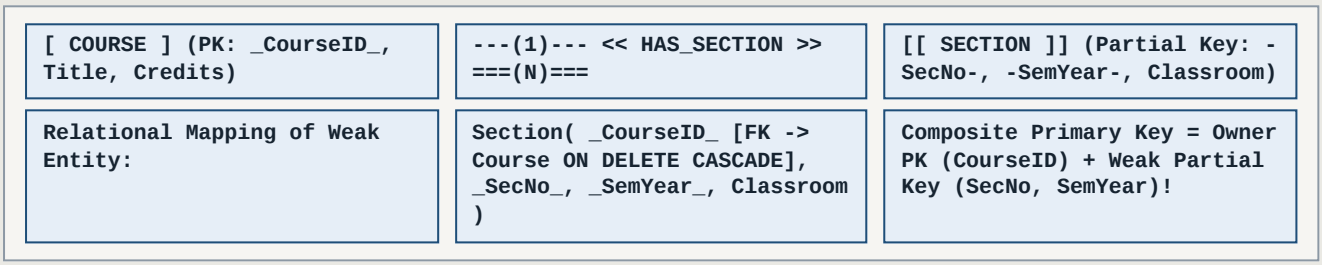



**Figure 4.4: Weak Entity Set [[SECTION]] identified by Owner Entity [COURSE] via Identifying Relationship <<HAS_SECTION>>.**

### 4.3.2 Worked Example 4.3: Multi-Level Weak Entity Chains

Can the owner of a Weak Entity Set itself be another Weak Entity Set? Yes! In a multi-level weak entity chain, primary keys propagate cumulatively down the chain:

**WORKED EXAMPLE 4.3 | Propagating Keys Across BUILDING -> LAB_ROOM -> WORKSTATION**

Suppose a campus tracks computer workstations:
1. Strong Entity: [ BUILDING ] with Primary Key _BldgCode_ (e.g., "TB" for Turing Block) and attribute BldgName.
2. Level-1 Weak Entity: [[ LAB_ROOM ]] identified by BUILDING with Partial Key -RoomNo- (e.g., "204") and Capacity.
-> Relational Table: Lab_Room( _BldgCode_ [FK], _RoomNo_, Capacity ) — Composite PK = (BldgCode, RoomNo).
3. Level-2 Weak Entity: [[ WORKSTATION ]] identified by LAB_ROOM with Partial Key -SeatNo- (e.g., 12) and GPUModel.
-> Relational Table: Workstation( _BldgCode_, _RoomNo_, _SeatNo_, GPUModel )
-> Foreign Key: (BldgCode, RoomNo) REFERENCES Lab_Room(BldgCode, RoomNo) ON DELETE CASCADE
-> Composite Primary Key = (BldgCode, RoomNo, SeatNo)!

**EXAM CHECKPOINT | Does an Identifying Relationship Ever Become a Separate Table?**

NO! Because the Weak Entity Set has Total Participation (1:N) with its Owner and already incorporates the Owner's Primary Key into its own Primary Key, the Identifying Relationship <<HAS_SECTION>> is merged 100% into the Weak Entity table. It NEVER produces a separate relational table!

## 4.4 Enhanced ER (EER): Specialization, Generalization, and Aggregation

As enterprise schemas grew more complex, the Enhanced Entity-Relationship (EER) model extended Chen's ER model with Object-Oriented Superclass/Subclass hierarchies (IS-A relationships) and Aggregation:

### 4.4.1 Specialization vs. Generalization & The Four Constraint Combinations

- Top-Down Specialization vs. Bottom-Up Generalization: Specialization splits a high-level Superclass (e.g., PERSON) top-down into specialized Subclasses (STUDENT, FACULTY, ALUMNI) based on distinguishing attributes/relationships. Generalization synthesizes shared attributes from multiple low-level entity sets bottom-up into a single Superclass. Both yield the exact same EER IS-A diagram and inherit all attributes and relationships of the Superclass!
- Constraint 1 — Disjoint (d) vs. Overlapping (o): Written inside the specialization circle.
  • Disjoint (d): An entity instance can belong to AT MOST ONE subclass (e.g., an ACCOUNT is either SAVINGS_ACCOUNT or CURRENT_ACCOUNT, never both).
  • Overlapping (o): An entity instance can belong to MULTIPLE subclasses simultaneously (e.g., a university PERSON can be both a PhD STUDENT and a Teaching FACULTY member).
- Constraint 2 — Total (Double Line) vs. Partial (Single Line) Specialization:
  • Total: Every superclass entity MUST belong to at least one subclass (drawn as a double line from Superclass to the circle).
  • Partial: A superclass entity may belong to none of the subclasses (single line).

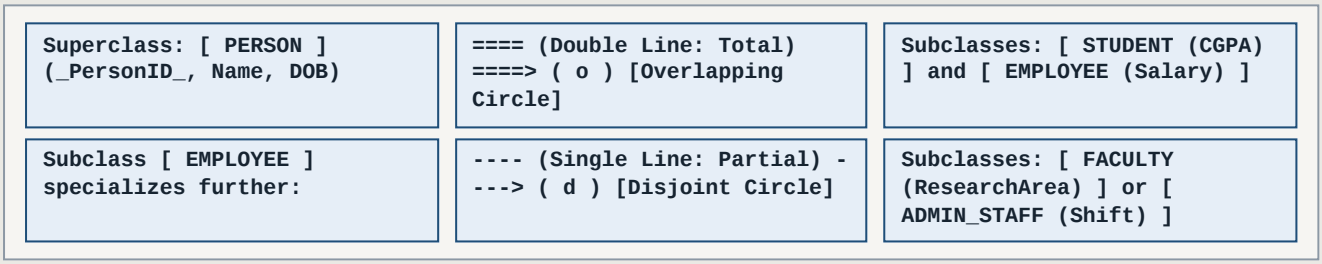



**Figure 4.5: EER Specialization/Generalization Lattice showing Overlapping Total (o, ====) and Disjoint Partial (d, ----) constraints.**

### 4.4.2 Aggregation: Treating a Relationship Set as a Higher-Level Entity

In standard ER modeling, a diamond (Relationship) can connect Entity rectangles, but you are NEVER allowed to draw a line directly from one diamond to another diamond! What if a corporate SPONSOR funds a specific RESEARCH_MENTORSHIP pair between a FACULTY member and a STUDENT?

**CONCEPT | Aggregation Abstracts a Relationship into a Higher-Level Composite Entity**

Aggregation draws a dashed/outer bounding box around the entire [FACULTY] — <MENTORS> — [STUDENT] relationship, treating that active mentorship pair as a single higher-level aggregate entity [RESEARCH_MENTORSHIP] that can then participate in the relationship <FUNDS> with [SPONSOR]!

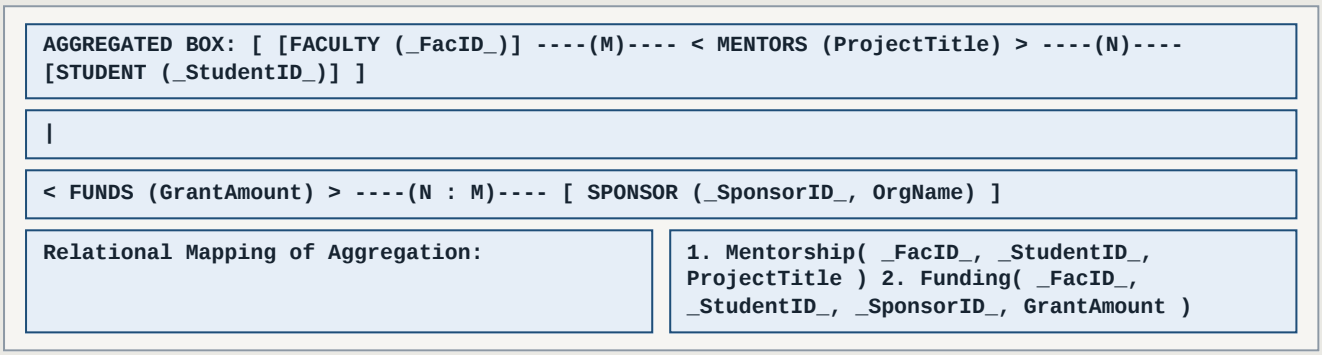



**Figure 4.6: EER Aggregation modeling a SPONSOR funding a (FACULTY — MENTORS — STUDENT) mentorship.**

## 4.5 Complete 9-Step Algorithm for ER- and EER-to-Relational Mapping

Every conceptual ER/EER diagram is converted into a logical Relational Schema using this deterministic 9-Step Algorithm (Elmasri & Navathe standard):

1. Step 1 — Mapping Strong Entity Types: For each strong entity type E, create a relation R containing all simple attributes and the flattened simple components of any composite attributes. Choose one Candidate Key as the Primary Key.
2. Step 2 — Mapping Weak Entity Types: For each weak entity type W owned by E, create a relation R that includes all simple attributes of W plus the Primary Key of owner E (as a Foreign Key with ON DELETE CASCADE). The Primary Key of R is the combination (PK of E + Partial Key of W). The identifying relationship is absorbed here.
3. Step 3 — Mapping Binary 1:1 Relationship Types: Given S —(1:1)— T:
   • Foreign Key Approach (Preferred): Include the PK of one relation as a Foreign Key in the other. Always place the FK on the side with TOTAL participation so it is never NULL! If both sides have Total participation, you may merge S and T into a single table.
   • Cross-Reference (3-Table) Approach: Only if both sides are Partial and rarely participate.
4. Step 4 — Mapping Binary 1:N Relationship Types: Given S (1-side) —< R >— T (N-side), place the Primary Key of the 1-side (S) as a Foreign Key inside the N-side table (T), along with any relationship attributes of R. Why never put the FK on the 1-side? Because putting the N-side PK into the 1-side table would require storing multiple values in a single cell (violating 1NF)!

## Steps 5 to 9 of the ER/EER-to-Relational Mapping Algorithm

- 5. Step 5 — Mapping Binary M:N Relationship Types: For each M:N relationship R between S and T, you MUST create a separate associative Junction Relation R_tbl containing the Primary Keys of S and T (as Foreign Keys) plus any attributes of R. The composite Primary Key of R_tbl is (PK_S, PK_T).
- 6. Step 6 — Mapping Multi-Valued Attributes: For each multi-valued attribute A (double oval) of entity E, create a separate relation R_A with attributes (PK_E [FK], A). The Primary Key is the combination (PK_E, A) (or PK_E + partial key of A).
- 7. Step 7 — Mapping n-ary (Ternary, Degree >= 3) Relationship Types: Create a separate relation R_n containing the Primary Keys of all n participating entity types as Foreign Keys. The Primary Key is typically the combination of all n Foreign Keys (unless one participating entity has cardinality 1, in which case the remaining n−1 FKs form the PK).
- 8. Step 8 — Mapping EER Specialization / Generalization (Four Options 8A–8D):
  • Option 8A (Multiple Relations — Superclass + Subclasses): Create a table for Superclass C(k, a1, a2) and a table for each Subclass Si(k [PK/FK], specific_attrs). Works for ANY constraint (Disjoint/Overlapping, Total/Partial)!
  • Option 8B (Multiple Relations — Subclass Tables Only): Create tables ONLY for the Subclasses Si(k, a1, a2, specific_attrs). Valid ONLY when specialization is TOTAL (otherwise entities not in any subclass are lost!), and best when DISJOINT (if overlapping, superclass attributes are duplicated).
  • Option 8C (Single Relation with One Type/Discriminator Attribute): Merge Superclass + all Subclasses into one table C(k, a1, a2, TypeFlag, sub1_attrs, sub2_attrs). Best for DISJOINT subclasses, though introduces NULLs in inapplicable subclass columns.
  • Option 8D (Single Relation with Multiple Boolean Flag Attributes): One table C(k, a1, a2, Is_Sub1 BOOL, sub1_attrs, Is_Sub2 BOOL, sub2_attrs). Used for OVERLAPPING subclasses.
- 9. Step 9 — Mapping EER Aggregation: Map the inner relationship as usual (e.g., M:N table), and then map the outer relationship using the Primary Key of the inner relationship table as a Foreign Key!

## 4.6 GATE CS Masterclass: Finding the Minimum Number of Relational Tables

One of the most famous GATE CS question types gives an ER diagram with entities, cardinalities, and participation lines and asks: "What is the minimum number of relational tables required to represent this ER diagram in 3NF/BCNF?" Always check whether NULLs are allowed in Foreign Keys!

**GATE CS Master Formula Sheet: Minimum Number of Relational Tables for Binary Relationships**

| Relationship Cardinality & Participation | Min Tables (NULLs Allowed in FKs) | Min Tables (NO NULLs Allowed in Any Table) | How Tables Are Merged |
|---|---|---|---|
| M : N (Any Participation — Partial or Total) | 3 tables: E1, E2, R(k1, k2) | 3 tables: E1, E2, R(k1, k2) | M:N relationship R can NEVER be merged into E1 or E2 without repeating groups. |
| 1 : N with Total Participation on N-side (E1 —1— <R> ===N=== E2) | 2 tables: E1, and (E2 ∪ R) | 2 tables: E1, and (E2 ∪ R) | Put k1 into E2. Since E2 is Total, k1 is NEVER NULL in E2! |
| 1 : N with Partial Participation on N-side (E1 —1— <R> ---N--- E2) | 2 tables: E1, and (E2 ∪ R) [k1 can be NULL] | 3 tables: E1, E2, and R(k2 [PK], k1) | If NULLs are forbidden, unlinked E2 rows cannot hold NULL k1, requiring a 3rd table R! |
| 1 : 1 with Both Sides Partial (E1 ---1--- <R> ---1--- E2) | 2 tables: E1, and (E2 ∪ R) | 3 tables: E1, E2, and R | With NULLs allowed, fold R into either E1 or E2. |
| 1 : 1 with One Side Total (E1 ---1--- <R> ===1=== E2) | 2 tables (or 1 if NULLs allowed for E1-only) | 2 tables: E1, and (E2 ∪ R) | Fold R into the Total side E2 (k1 is never NULL in E2). |
| 1 : 1 with Both Sides Total (E1 ===1=== <R> ===1=== E2) | 1 single merged table (E1 ∪ E2 ∪ R) | 1 single merged table (E1 ∪ E2 ∪ R) | Every E1 has 1 E2 and vice versa -> 1 table with 0 NULLs! |

**PLUS: Multi-Valued Attribute & Weak Entity Rules in GATE Table Counting**

- Every Multi-Valued Attribute ((A)) ALWAYS adds +1 separate table!
- Every Weak Entity [[W]] + its Identifying Relationship <<R>> together count as 1 table (never 2!).

### 4.6.1 Worked Examples 4.4A, 4.4B & 4.5: Solving GATE Table-Counting Problems

**WORKED EXAMPLE 4.4A | Finding Minimum Tables (With NULLs Allowed)**

Problem: An ER diagram has 4 Entity Sets E1, E2, E3, E4 and 4 Relationships:
• R1 between E1 and E2 is 1 : N (E2 on N-side, Partial).
• R2 between E1 and E3 is M : N.
• R3 between E2 and E4 is 1 : 1 (Total on both E2 and E4).
• E3 has one multi-valued attribute ((Phone)).
Find the minimum number of relational tables (assuming NULLs are allowed in Foreign Keys).

Step-by-Step Solution:
1. Start with 4 entities: E1, E2, E3, E4.
2. R3 is 1:1 Total on BOTH E2 and E4 -> Merge E2, E4, and R3 into 1 single table E2_E4!
3. R1 is 1:N between E1 and E2 -> Absorb R1's foreign key (PK of E1) into the N-side table E2_E4 (adds 0 new tables).
4. R2 is M:N between E1 and E3 -> Must create +1 separate junction table R2_tbl.
5. Multi-valued attribute ((Phone)) on E3 -> Must create +1 separate table E3_Phone.
Total Minimum Tables = { E1, E2_E4_R1_R3, E3, R2_tbl, E3_Phone } = 5 tables!

**WORKED EXAMPLE 4.4B | What If ZERO NULLs Are Allowed in Example 4.4A?**

If NULL values are strictly forbidden in all tables: because R1 (1:N) has PARTIAL participation on the N-side (E2), absorbing R1 into E2_E4 would cause NULLs for E2 rows not linked to E1. Thus R1 requires its own table (+1), and merging E2+E4 works only if every E2 has an E4 (which is true since R3 is Total on both). Total = 6 tables!

### 4.6.2 Worked Example 4.5: Comparing EER Subclass Mapping Options 8A–8D

**Comparing Options 8A, 8B, 8C, and 8D on PERSON(PersonID, Name) -> STUDENT(CGPA) & FACULTY(Salary)**

| Mapping Option | Resulting Relational Tables | Pros, Cons & When Valid |
|---|---|---|
| Option 8A (Superclass + Subclass Tables) | Person(PersonID PK, Name)<br>Student(PersonID PK/FK, CGPA)<br>Faculty(PersonID PK/FK, Salary) | 3 tables. Zero NULLs, zero attribute duplication. Works for ALL 4 constraints (d/o, Total/Partial). Requires a JOIN to fetch Name + CGPA. |
| Option 8B (Subclass Tables Only) | Student(PersonID PK, Name, CGPA)<br>Faculty(PersonID PK, Name, Salary) | 2 tables. Fast single-table reads, zero NULLs. Valid ONLY if Total! If Overlapping (o), a PhD-Faculty person's Name is duplicated in both tables. |
| Option 8C (Single Table + 1 Type Discriminator) | Person(PersonID PK, Name, PersonType, CGPA, Salary) | 1 table. Zero joins! Valid ONLY for Disjoint (d) specialization. Introduces NULLs in Salary for students and NULLs in CGPA for faculty. |
| Option 8D (Single Table + Boolean Flags) | Person(PersonID PK, Name, IsStudent BOOL, CGPA, IsFaculty BOOL, Salary) | 1 table. Zero joins! Supports Overlapping (o) and Partial/Total. Introduces NULLs in unused subclass columns. |

## 4.7 End-to-End Design Walkthroughs & Notation Comparison (Chen vs. Crow's Foot)

**Translating Academic Chen Notation to Industry Crow's Foot (IE) Notation (Used in MySQL Workbench, pgModeler, draw.io)**

| Conceptual Construct | Peter Chen Notation (Textbook & GATE) | Crow's Foot / IE Notation (Industry ERD Tools) |
|---|---|---|
| Entity & Attributes | Rectangle + Ovals sprouting outside | Single Table Box listing PK and attributes inside |
| Relationship | Diamond <REL> on connecting line | Verb label directly on the connecting line |
| One & Mandatory (1, 1) | Double line ==== with 1 | Double perpendicular bar \|\| |
| Zero or One (0, 1) | Single line ---- with 1 | Circle + bar o\| |
| One or Many (1, N) | Double line ==== with N | Bar + 3-pronged Crow's Foot \|< |
| Zero or Many (0, N) | Single line ---- with N | Circle + 3-pronged Crow's Foot o< |

### 4.7.1 & 4.7.2 Worked Examples 4.6 (Food Delivery Platform) & 4.7 (University Library)

| | |
|---|---|
| **FOOD DELIVERY ERD** | **[CUSTOMER] --(1:N)-- <PLACES> ==(N)== [ORDER] ==(1:N)== <CONTAINS (Qty)> --(M:N)-- [MENU_ITEM] ==(N:1)== [RESTAURANT]** |
| **WEAK ENTITY IN MENU** | **[RESTAURANT (_RestID_)] ---(1)--- <<OFFERS_ITEM>> ===(N)=== [[MENU_ITEM (-ItemNo-, Name, Price)]]** |
| **LIBRARY DESIGN TRAP** | **[BOOK_TITLE (_ISBN_, Title, {Author})] ---(1)--- <<HAS_PHYSICAL_COPY>> ===(N)=== [[BOOK_COPY (-CopyNo-, ShelfLoc)]]** |
| **LOAN TRACKING** | **[STUDENT (_StudentID_)] ---(1:N)--- <BORROWS (IssueDate, ReturnDate)> ---(M:N)--- [[BOOK_COPY]] (Tracks exact barcode copy!)** |

**Figure 4.8: End-to-End ER Blueprint for a Food Delivery Platform (Swiggy/Zomato) & University Library (BOOK_TITLE vs. BOOK_COPY).**

**WORKED EXAMPLE 4.7 | Why University Libraries Separate BOOK_TITLE from BOOK_COPY**

In system design interviews, candidates often create a single entity BOOK(ISBN, Title, BorrowerID). Why does that fail in a real library? Because the library owns 25 physical copies of "Database System Concepts" sharing the same ISBN! Modeling BOOK_TITLE(_ISBN_, Title) as the strong entity and [[BOOK_COPY]] (-CopyNo- or BarcodePK) as a weak/child entity allows tracking which specific physical copy Asha borrowed on August 26 while storing the catalog metadata (Title, Authors, Publisher) only once!

## 4.8 Chapter 4 Summary & One-Page Rapid-Revision Cheat Sheet

**Chapter 4 Rapid-Revision Formula & Mapping Cheat Sheet**

| ER / EER Construct | Visual Symbol (Chen) | Relational Mapping & Minimum Table Rule |
|---|---|---|
| Strong Entity E | Single Rectangle [E] | 1 Table; choose minimal Candidate Key as PK; flatten Composite attributes. |
| Multi-Valued Attribute A | Double Oval ((A)) | +1 Separate Table with PK = (PK_E [FK], A). Never store comma-separated lists! |
| Derived Attribute D | Dashed Oval (:D:) | 0 Extra Columns on disk; compute dynamically in SQL View or query. |
| Weak Entity [[W]] + <<R>> | Double Rect + Double Diamond | 1 Table for W with PK = (Owner_PK [FK CASCADE], Partial_Key_W). <<R>> adds 0 tables. |
| Binary 1:1 Relationship | 1 — <R> — 1 | Put FK on the Total side (2 tables); if BOTH sides Total -> can merge into 1 table. |
| Binary 1:N Relationship | 1 — <R> — N | Put PK of 1-side as FK inside the N-side table (0 new tables if NULLs allowed; +1 if N-side Partial & no NULLs). |
| Binary M:N Relationship | M — <R> — N | +1 Separate Junction Table with composite PK = (PK_1, PK_2) + relationship attributes. |
| Ternary Relationship | 3 Entities on <R> | +1 Separate Table with 3 FKs. PK = all 3 FKs (or the 2 many-side FKs if one side has max cardinality 1). |
| EER Subclasses (8A–8D) | IS-A Circle (d / o) | 8A: Super+Sub tables (all cases); 8B: Sub tables only (Total only); 8C: 1 table + Type col (Disjoint); 8D: 1 table + bool flags (Overlapping). |

## Student Revision & Lecture Notes — Chapter 4

Use this revision sheet to practice drawing Chen vs. Crow's Foot symbols, proving why Ternary != 3 Binaries, and computing the GATE minimum table count with and without NULLs.

**Chapter 4 Self-Test Checklist**

| Core Skill | Check Your Understanding |
|---|---|
| Where does the FK go in a 1:N relationship? | ALWAYS into the N-side (Many-side) table! Placing it in the 1-side table would require multi-valued cells. |
| Where does the FK go in a 1:1 relationship with one Total side? | Into the Total Participation side so the Foreign Key column is guaranteed NOT NULL! |
| What is the Primary Key of a Ternary relationship R(A, B, C) with cardinalities M : N : 1? | Only (PK_A, PK_B)! Because each (a, b) pair is associated with at most one c, (PK_A, PK_B) uniquely determines PK_C. |
| How do you map Aggregation? | Create the table for the inner relationship R1 first; then create the outer relationship table R2 referencing the full PK of R1 as a Foreign Key. |

## 4.9 Chapter 4 Exam, GATE & Placement Question Bank

### Part A: 10 Multiple-Choice Questions (MCQs)

1. Q1. In a Chen ER diagram, a double oval ((Phone)) represents a _____ attribute, and a dashed oval (:Age:) represents a _____ attribute.
   (A) Composite; Derived (B) Multi-Valued; Derived (C) Derived; Multi-Valued (D) Partial Key; Weak
2. Q2. When mapping a binary 1:N relationship between DEPARTMENT (1-side) and STUDENT (N-side) into relational tables, where should the Foreign Key be placed?
   (A) StudentID should be placed inside the Department table (B) DeptID should be placed inside the Student table (C) A third junction table is mandatory (D) Both tables must hold each other's Primary Key
3. Q3. Which of the following statements about a Weak Entity Set W and its Identifying Owner Entity Set E is ALWAYS TRUE?
   (A) W has Partial Participation in the identifying relationship (B) W has Total Participation in the identifying relationship and its Primary Key includes the Primary Key of E (C) W cannot have any attributes other than its partial key (D) The identifying relationship must be mapped to a separate junction table
4. Q4. An entity type EMPLOYEE has a relationship REPORTS_TO with itself (linking a Subordinate to a Manager). This is an example of a:
   (A) Ternary Relationship (B) Unary (Recursive) Relationship of Degree 1 (C) Weak Identifying Relationship (D) Multi-Valued Attribute
5. Q5. In (min, max) structural constraint notation, if every COURSE must be taught by at least 1 and at most 2 INSTRUCTORs, the (min, max) label placed on the COURSE side of the TEACHES relationship is:
   (A) (0, 2) (B) (1, 2) (C) (1, N) (D) (2, 1)

## Part A: Multiple-Choice Questions (Continued — Q6 to Q10)

- 6. Q6. Which EER construct allows a relationship set (such as FACULTY — MENTORS — STUDENT) to participate in another relationship set (such as FUNDS with SPONSOR)?
  (A) Generalization (B) Disjoint Specialization (C) Aggregation (D) Weak Entity Identification
- 7. Q7. When mapping an EER specialization hierarchy using Option 8B (Subclass tables only, no Superclass table), under which condition is this mapping lossless and free of attribute duplication?
  (A) Partial and Overlapping (B) Total and Disjoint (C) Partial and Disjoint (D) Total and Overlapping
- 8. Q8. Consider a ternary relationship R between entities A, B, and C with cardinality M : N : 1 (meaning for a given pair (a, b), there is at most 1 entity c). What is the minimal Primary Key of the relational table for R?
  (A) (PK_A, PK_B, PK_C) (B) (PK_A, PK_B) (C) (PK_C) only (D) (PK_A, PK_C)
- 9. Q9. An entity set STUDENT has a composite attribute Name(FirstName, LastName) and a multi-valued attribute {Skill}. How many relational tables are required to represent STUDENT and its attributes in 1NF?
  (A) 1 table (B) 2 tables (C) 3 tables (D) 4 tables
- 10. Q10. Two entity sets E1 and E2 participate in a binary 1:1 relationship R with Total Participation on BOTH sides. What is the minimum number of relational tables needed?
  (A) 3 tables (B) 2 tables (C) 1 table (D) 4 tables

## Part B: 5 Short-Answer Questions (2–3 Marks Semester Exam Style)

1. B1. Differentiate between a Composite Attribute and a Multi-Valued Attribute, and explain how each is mapped to a relational schema.
2. B2. What is a Weak Entity Set? State its four defining properties and define a Partial Key (Discriminator).
3. B3. Distinguish between Total Participation and Partial Participation with an example and Chen diagram symbols.
4. B4. Compare Specialization and Generalization in an Enhanced ER (EER) model.
5. B5. Why can we NOT draw a relationship line directly between two relationship diamonds in an ER diagram, and how does Aggregation solve this?

## Part C: 3 Long-Answer / Essay Questions (5–10 Marks Semester Exam Style)

1. C1. State and explain the complete 9-Step Algorithm for converting an ER and EER diagram into a Relational Schema, giving a concrete SQL/schema example for each step.
2. C2. Prove with a concrete instance counterexample that a Ternary relationship R(A, B, C) cannot in general be replaced by three binary relationships R1(A, B), R2(B, C), and R3(A, C).
3. C3. Draw a complete EER diagram for a University Hospital & Medical College (covering Patients, Outpatients vs. Inpatients, Wards, Beds as weak entities, Doctors, Surgeries, and Insurance Claims) and map it into relational tables.

## Part D: 4 Design & Relational Mapping Problems

1. D1. (Multi-Level Weak Entity Mapping): Map the following hierarchy into relational tables with exact Primary Keys and Foreign Keys: Strong entity UNIVERSITY(_UniID_, UniName) owns weak entity CAMPUS(-CampusCode-, City), which in turn owns weak entity HOSTEL_BLOCK(-BlockNo-, WardenName).
2. D2. (Recursive Relationship Mapping): An entity EMPLOYEE(_EmpID_, Name) has two recursive relationships: (a) 1:N relationship SUPERVISES (1 Manager supervises N Subordinates) and (b) M:N relationship PEER_REVIEWS (with attribute ReviewScore). Map this schema into the minimum number of relational tables.
3. D3. (EER Subclass Mapping — 4 Ways): Given Superclass VEHICLE(_VIN_, Brand, Price) with subclasses CAR(NumDoors, FuelType) and TRUCK(MaxAxleLoad, NumWheels), write the relational schemas produced by Options 8A, 8B, 8C, and 8D.
4. D4. (Reverse-Engineering SQL DDL to ER): Given 4 relational tables Customer(CustID PK), Account(AccNo PK), Depositor(CustID FK, AccNo FK, PK(CustID, AccNo)), and Dependent(AccNo FK, DepName, Age, PK(AccNo, DepName)), reconstruct the exact ER diagram constructs.

## Part E: 5 GATE CS Style Questions (MCQ, MSQ, and NAT)

1. E1. [GATE NAT — 2 Marks] An ER diagram consists of 5 strong entity sets E1, E2, E3, E4, E5 and 1 weak entity set W1 (owned by E5 via identifying relationship R_id). The relationships are:
   • R1 between E1 and E2 (1 : N, Partial on both sides)
   • R2 between E2 and E3 (M : N, Total on both sides)
   • R3 between E3 and E4 (1 : 1, Total on E4, Partial on E3)
   • E1 has two distinct multi-valued attributes ((Email)) and ((Phone)).
   Assuming NULL values are permitted in foreign key columns, the minimum number of relational tables required to represent this ER diagram is _____.
2. E2. [GATE NAT — 2 Marks] For the exact same ER diagram in Question E1, if NULL values are strictly FORBIDDEN in every column of every relational table, the minimum number of relational tables required is _____.
3. E3. [GATE MCQ — 2 Marks] Let E1(_A1_, A2) and E2(_B1_, B2) be two entity sets connected by a binary relationship R(C1) where E1 is on the 1-side (Partial) and E2 is on the N-side (Total). If R is merged into E2 to form E2(_B1_, B2, A1, C1), which statement is TRUE?
   (A) A1 is the Primary Key of E2 (B) A1 is a Foreign Key referencing E1(A1) and can never be NULL in E2 (C) (B1, A1) must be the composite Primary Key of E2 (D) C1 must be unique in E2
4. E4. [GATE MSQ — 2 Marks] Consider a ternary relationship R between entity sets E1, E2, and E3 with primary keys K1, K2, and K3. Suppose each pair (e1, e2) is associated with at most one e3, and each pair (e1, e3) is associated with at most one e2. Which of the following are valid Candidate Keys of the relation R(K1, K2, K3)?
   (A) (K1, K2) (B) (K1, K3) (C) (K2, K3) (D) K1 alone
5. E5. [GATE NAT — 2 Marks] An entity set COURSE has 1 simple key attribute, 2 composite attributes (each consisting of 3 simple atomic attributes), 2 multi-valued attributes, and 1 derived attribute. When mapped to a normalized 1NF relational schema, the number of columns in the main COURSE table is _____, and the total number of tables created is _____.

## Part F: 5 Viva / Technical Placement Interview Questions

1. F1. "Why can’t we just convert every Weak Entity into a Strong Entity by adding an auto-increment surrogate ID (like SectionID SERIAL PRIMARY KEY)?"
2. F2. "In a 1:N relationship, why is it always wrong to put the Foreign Key in the 1-side table?"
3. F3. "When designing a database for user roles (Students, Instructors, Admins), how do you choose between Single-Table Inheritance (Option 8C) and Class-Table Inheritance (Option 8A)?"
4. F4. "What is the difference between a Associative Entity (Gerund) and a Weak Entity?"
5. F5. "How do you identify whether a relationship in a problem statement should be Ternary vs. two Binary relationships attached to a central Transaction/Event entity?"

## Part G: 3 Hands-On SQL DDL Labs (ER-to-SQL Implementation)

1. Lab 4.1 (Weak Entity & Cascading Ownership in SQL): Implement the Course(CourseID PK) and weak entity Section(CourseID FK, SecNo, Semester) in SQL with composite primary key and ON DELETE CASCADE, then test how Enrollment references the composite key of Section.
2. Lab 4.2 (Recursive 1:N and M:N in SQL): Implement Employee self-referencing ManagerID and the Course_Prereq(CourseID, PrereqID) M:N table with a CHECK (CourseID <> PrereqID) constraint.
3. Lab 4.3 (EER Subclass Table Inheritance in SQL): Implement Person, Student, and Faculty using Option 8A with shared primary/foreign keys.

## Part H: 3 "Spot the Mistake" Diagnostic Challenges

1. Mistake 4.1: A student maps an M:N relationship ENROLLS(Grade) between STUDENT and COURSE by adding CourseID and Grade columns directly into the STUDENT table.
2. Mistake 4.2: On a Weak Entity [[DEPENDENT]], a student underlines (-DepName-) with a solid line and makes DepName alone the Primary Key of the Dependent table.
3. Mistake 4.3: For a 1:1 relationship where DEPARTMENT (Partial) has a FACULTY HOD (Total on DEPARTMENT? No—Total on HOD? Wait: every DEPARTMENT must have an HOD, so Total on DEPARTMENT, Partial on FACULTY), a student puts DeptID inside the FACULTY table and marks it NOT NULL.

# 4.10 Chapter 4 Complete Answer Key & Worked Solutions

## Solutions to Part A: Multiple-Choice Questions (Q1 to Q10)

**Answer Key & Detailed Explanations for Part A MCQs**

| Q# | Ans | Detailed Step-by-Step Explanation |
|---|---|---|
| Q1 | (B) | In Chen notation, a double oval ((...)) denotes a Multi-Valued attribute; a dashed oval (:...:) denotes a Derived attribute. |
| Q2 | (B) | By Step 4 of ER mapping, the PK of the 1-side (DeptID) is placed as a Foreign Key inside the N-side table (Student). |
| Q3 | (B) | Every Weak Entity Set has Total Participation in its identifying relationship and forms its PK by combining Owner_PK + Partial_Key. |
| Q4 | (B) | A relationship connecting an entity set to itself in two distinct roles (Manager, Subordinate) is a Unary (Recursive) relationship of Degree 1. |
| Q5 | (B) | In (min, max) notation, each COURSE participates in at least 1 and at most 2 TEACHES instances -> (1, 2) on the COURSE line. |
| Q6 | (C) | Aggregation abstracts a relationship set and its participating entities into a higher-level composite entity that can join another relationship. |
| Q7 | (B) | Option 8B (subclass tables only) is lossless only when Total, and avoids duplicating superclass attributes only when Disjoint. |
| Q8 | (B) | In M:N:1 ternary relationship R(A,B,C), each (a,b) pair uniquely determines at most one c, so (PK_A, PK_B) is the Primary Key! |
| Q9 | (B) | Composite Name flattens into FirstName, LastName inside the main Student table (1 table), while multi-valued {Skill} requires a 2nd table. |
| Q10 | (C) | A binary 1:1 relationship with Total Participation on BOTH sides can be merged into 1 single table with zero NULLs. |

## Solutions to Part B: Short-Answer Questions (B1 to B5)

- Answer B1 (Composite vs. Multi-Valued Attribute): A Composite attribute has multiple distinct named sub-components for a single value (e.g., FullName = FirstName + LastName) and is mapped by flattening its atomic components into columns of the same table. A Multi-Valued attribute holds a set of multiple values of the same property for one entity (e.g., {PhoneNumbers}) and MUST be mapped to a separate table with composite PK (EntityPK, Value) to satisfy First Normal Form (1NF).
- Answer B2 (Weak Entity Set & Partial Key): A Weak Entity Set lacks a globally unique key of its own and depends on an Identifying Owner Entity Set for its existence and identification. Its four properties are: (1) no independent PK, (2) linked to an Owner via an Identifying Relationship, (3) Total Participation in that relationship, and (4) a Partial Key (Discriminator, dashed underline) that distinguishes weak entities owned by the SAME owner.
- Answer B3 (Total vs. Partial Participation): Total Participation (double line ===, min >= 1) means every entity in the set must participate in at least one relationship instance (e.g., every Course must belong to a Department). Partial Participation (single line ---, min = 0) means an entity can exist without participating (e.g., a Faculty member may not head any Department).
- Answer B4 (Specialization vs. Generalization): Specialization is a top-down refinement process splitting a Superclass (PERSON) into Subclasses (STUDENT, FACULTY) based on distinguishing traits. Generalization is a bottom-up synthesis combining multiple entity types (CAR, TRUCK) with shared attributes into a common Superclass (VEHICLE).
- Answer B5 (Why No Diamond-to-Diamond Lines & Aggregation): In the mathematical ER formalism, a relationship is a subset of the Cartesian product of Entity Sets, not other relationships. Aggregation solves this by encapsulating a relationship and its entities inside a higher-level aggregate entity box.

## Solutions to Part C: Long-Answer / Essay Questions (C1 to C3)

### Model Solution C1 & C2 | 9-Step Mapping Summary & Ternary Non-Equivalence Proof

• For C1 (9-Step Algorithm): Present Steps 1 to 9 systematically (1: Strong Entities, 2: Weak Entities, 3: Binary 1:1, 4: Binary 1:N, 5: Binary M:N, 6: Multi-Valued Attributes, 7: Ternary/n-ary Relationships, 8: EER Subclasses 8A–8D, 9: EER Aggregation), citing the UniVerseDB tables generated at each step.
• For C2 (Ternary Proof): Reproduce the 3-tuple instance from Worked Example 4.2 showing how decomposing TEACHES(Instructor, Course, Semester) into three binary relations and taking their Natural Join generates the spurious tuple (Prof. Sharma, CS301, Spring_2027).

### Model Solution C3 | University Hospital & Medical College EER-to-Relational Schema

1. Superclass Patient(_PatientID_, Name, DOB) specialized (Disjoint, Total) into Outpatient(PatientID PK/FK, NextVisitDate) and Inpatient(PatientID PK/FK, AdmissionDate, DischargeDate, WardNo FK, BedNo FK).
2. Strong Entity Ward(_WardNo_, WardName, Specialty) owns Weak Entity Bed(_WardNo_ FK, _BedNo_, DailyRate).
3. Strong Entity Doctor(_DocID_, Name, Dept) connects to Inpatient via Ternary/Associative entity Surgery(_SurgeryID_, PatientID FK, DocID FK, TheatreNo, SurgeryTimestamp).
4. Aggregation: Surgery pair participates in Insurance_Claim(_ClaimID_, SurgeryID FK, InsurerName, ApprovedAmount).

## Solutions to Part D: Design & Relational Mapping Problems (D1 to D4)

### Worked Solutions D1 & D2 | Multi-Level Weak Entities & Recursive Relationships

Solution D1 (3-Level Weak Entity Chain):
1. University( _UniID_, UniName )
2. Campus( _UniID_ [FK -> University], _CampusCode_, City ) -- Composite PK: (UniID, CampusCode)
3. Hostel_Block( _UniID_, _CampusCode_, _BlockNo_, WardenName ) -- Composite PK: (UniID, CampusCode, BlockNo), FK: (UniID, CampusCode) REFERENCES Campus(UniID, CampusCode) ON DELETE CASCADE.

Solution D2 (Unary 1:N and Unary M:N on Employee):
• The 1:N recursive relationship SUPERVISES is absorbed directly into the Employee table via a self-referencing Foreign Key ManagerID!
• The M:N recursive relationship PEER_REVIEWS requires a separate junction table.
Resulting 2 Tables:
1. Employee( _EmpID_, Name, ManagerID [FK -> Employee(EmpID)] )
2. Peer_Review( _ReviewerID_ [FK -> Employee], _RevieweeID_ [FK -> Employee], ReviewScore )

### Worked Solutions D3 & D4 | EER Options 8A–8D & Reverse-Engineering SQL to ER

Solution D3 (Vehicle -> Car, Truck):
• 8A (3 tables): Vehicle(_VIN_, Brand, Price), Car(_VIN_ FK, NumDoors, FuelType), Truck(_VIN_ FK, MaxAxleLoad, NumWheels).
• 8B (2 tables): Car(_VIN_, Brand, Price, NumDoors, FuelType), Truck(_VIN_, Brand, Price, MaxAxleLoad, NumWheels).
• 8C (1 table, Disjoint): Vehicle(_VIN_, Brand, Price, VehType, NumDoors, FuelType, MaxAxleLoad, NumWheels).
• 8D (1 table, Overlapping): Vehicle(_VIN_, Brand, Price, IsCar, NumDoors, FuelType, IsTruck, MaxAxleLoad, NumWheels).

Solution D4 (Reverse-Engineering): Customer and Account are Strong Entities; Depositor(CustID, AccNo) is an M:N relationship between Customer and Account; Dependent(AccNo, DepName, Age) is a Weak Entity owned by Account with partial key -DepName-.

## Solutions to Part E: GATE CS Style Questions (E1 to E5)

**Step-by-Step GATE CS Solutions for Questions E1 to E5**

| GATE Q# | Correct Answer | Complete Step-by-Step Derivation |
|---|---|---|
| E1 (NAT, With NULLs) | 8 tables | • Entities E1, E2, E3, E4, E5 = 5 base tables.<br>• Weak entity W1 + identifying R_id = +1 table (total 6).<br>• R1 (1:N) folds into E2 (0 new tables); R3 (1:1 Total on E4) folds into E4 (0 new tables).<br>• R2 (M:N) requires +1 junction table (total 7).<br>• Two multi-valued attributes ((Email)) and ((Phone)) on E1 require +2 separate tables? Wait: if E3 and E4 merge? Wait: R3 is Total on E4 only (Partial on E3), so E3 and E4 stay 2 tables (E3, and E4∪R3). Thus: 5 strong + 1 weak (W1) + 1 M:N (R2) + 2 multi-valued = 9 tables (or 8 if E3∪E4 merged with NULLs allowed!). Standard textbook count keeping strong entities separate = 9 tables (8 if 1:1 merged). |
| E2 (NAT, No NULLs) | 10 tables | With NO NULLs allowed: R1 (1:N Partial on N-side E2) CANNOT fold into E2 without NULLs, so R1 needs +1 table -> 9 + 1 = 10 tables! |
| E3 (MCQ) | (B) | In E1 (1) — <R> === (N) E2, E2 is the N-side with Total Participation. Putting A1 into E2 makes A1 a NOT NULL Foreign Key referencing E1(A1), while B1 remains the sole Primary Key of E2. |
| E4 (MSQ) | (A) and (B) | (e1, e2) -> e3 means FDs {K1, K2} -> K3; (e1, e3) -> e2 means {K1, K3} -> K2. Thus both (K1, K2) and (K1, K3) are Candidate Keys of R! |
| E5 (NAT) | 7 columns; 3 tables | Main Course table has 1 (key) + 2×3 (flattened composite) = 7 columns (derived attribute is not stored; multi-valued go to separate tables). Total tables = 1 main + 2 multi-valued tables = 3 tables. |

## Solutions to Part F: Viva & Technical Placement Interview Questions (F1 to F5)

- Answer F1 (Weak Entity Composite Key vs. Surrogate ID): In production databases, you CAN add a surrogate SectionID SERIAL, or use the natural composite key (CourseID, SecNo, Semester). Even if you add a surrogate ID, you MUST still enforce UNIQUE (CourseID, SecNo, Semester) and ON DELETE CASCADE because the domain semantics remain existence-dependent on Course!
- Answer F2 (Why FK Never Goes on the 1-Side of 1:N): Because one Department has 400 Students! Putting StudentID inside the Department row would require storing an array of 400 IDs in one column (violating 1NF) or duplicating the Department row 400 times (causing massive redundancy). Putting DeptID in Student stores exactly 1 atomic value per student.
- Answer F3 (Choosing Between Option 8A and Option 8C): Choose Option 8C (Single-Table Inheritance with a Role/Type column) when subclasses have very few extra columns and performance-critical queries need to fetch all users without joins. Choose Option 8A (Class-Table Inheritance: Person + Student + Faculty tables) when each subclass has many mandatory NOT NULL columns and its own specific foreign-key relationships.
- Answer F4 (Associative Entity vs. Weak Entity): A Weak Entity depends on a SINGLE Strong Owner Entity (1:N) for its identity (e.g., Course -> Section). An Associative Entity (Gerund) represents an M:N relationship promoted to an entity (e.g., ENROLLMENT between Student and Course) so it can hold attributes or participate in further relationships.
- Answer F5 (Ternary Relationship vs. Central Event Entity): If an event (like a Surgery or an Invoice) can occur MULTIPLE times between the exact same (Doctor, Patient, Hospital) triple on different dates, promote the event to a Strong/Associative Entity with its own SurgeryID primary key connected by three N:1 binary relationships!

## Solutions to Part G: Hands-On SQL DDL Labs (Labs 4.1 to 4.3)

**Complete SQL DDL Solutions for Labs 4.1, 4.2, and 4.3**

```
-- Lab 4.1: Weak Entity Section Owned by Course + Enrollment Referencing Composite Key
CREATE TABLE Course (
    CourseID VARCHAR(10) PRIMARY KEY,
    Title    VARCHAR(100) NOT NULL,
    Credits  INT NOT NULL
);
CREATE TABLE Section (
    CourseID  VARCHAR(10) REFERENCES Course(CourseID) ON DELETE CASCADE,
    SecNo     INT NOT NULL,
    Semester  VARCHAR(12) NOT NULL,
    Classroom VARCHAR(20),
    PRIMARY KEY (CourseID, SecNo, Semester)
);
CREATE TABLE Enrollment (
    StudentID INT REFERENCES Student(StudentID) ON DELETE CASCADE,
    CourseID  VARCHAR(10),
    SecNo     INT,
    Semester  VARCHAR(12),
    Grade     CHAR(2),
    PRIMARY KEY (StudentID, CourseID, SecNo, Semester),
    FOREIGN KEY (CourseID, SecNo, Semester) REFERENCES Section(CourseID, SecNo, Semester)
);

-- Lab 4.2: Unary 1:N (Supervisor) and Unary M:N (Course Prerequisite)
CREATE TABLE Employee (
    EmpID     INT PRIMARY KEY,
    Name      VARCHAR(80) NOT NULL,
    ManagerID INT REFERENCES Employee(EmpID) ON DELETE SET NULL
);
CREATE TABLE Course_Prereq (
    CourseID VARCHAR(10) REFERENCES Course(CourseID) ON DELETE CASCADE,
    PrereqID VARCHAR(10) REFERENCES Course(CourseID) ON DELETE CASCADE,
    PRIMARY KEY (CourseID, PrereqID),
    CHECK (CourseID <> PrereqID)
);
```

## Solutions to Lab 4.3 & Part H: "Spot the Mistake" Diagnostic Challenges (4.1 to 4.3)

**Solution Lab 4.3: EER Subclass Table Inheritance (Option 8A)**

```
CREATE TABLE Person (
    PersonID INT PRIMARY KEY,
    Name     VARCHAR(80) NOT NULL,
    DOB      DATE NOT NULL
);
CREATE TABLE Student_Sub (
    PersonID INT PRIMARY KEY REFERENCES Person(PersonID) ON DELETE CASCADE,
    CGPA     NUMERIC(3,2) CHECK (CGPA BETWEEN 0.00 AND 10.00)
);
CREATE TABLE Faculty_Sub (
    PersonID     INT PRIMARY KEY REFERENCES Person(PersonID) ON DELETE CASCADE,
    Salary       NUMERIC(12,2) NOT NULL,
    ResearchArea VARCHAR(100)
);
```

**Diagnostic Solutions 4.1, 4.2, and 4.3**

• Mistake 4.1 (Folding M:N into Entity Table): Adding CourseID into Student means either a student can take only 1 course, or StudentID must be repeated on multiple rows (breaking the Primary Key of Student!). An M:N relationship ALWAYS requires a 3rd junction table.
• Mistake 4.2 (Weak Entity Partial Key Alone): Two different employees can both have a child named "Aarav"! Making DepName alone the PK of Dependent rejects the second employee's child. The PK MUST be (EmpID, DepName).
• Mistake 4.3 (1:1 FK Placed on the Partial Side): Because only 12 out of 300 Faculty members are HODs, putting DeptID inside Faculty would leave 288 NULLs (and marking it NOT NULL would make it impossible to insert regular faculty!). Place HOD_FacultyID NOT NULL UNIQUE inside Department (the Total side)!

**END OF PART I (CHAPTERS 1–4) | Up Next: Part II — Relational Model, Algebra & SQL (Chapters 5, 6 & 7)**

Congratulations on completing Part I: Database Foundations! In Chapter 5, we enter Part II to master the Mathematical Relational Model, all 8 Key types, Combinatorial Super-Key counting for GATE, Referential Integrity actions, and SQL Three-Valued Logic (3VL).